FINAL PROJECT SUMMARY: JavaFX To-Do List Application

Student Name: Berra Erkol **Signature:**

Student ID: 25120102007

GitHub Link: https://github.com/berraerkol/ToDoListApp-JavaFX-Final

1. Project Overview

For my final project, I developed a desktop To-Do List application. I used the Java programming language and the JavaFX framework for the graphical user interface (GUI). The main goal of this project is to help users organize their daily tasks easily and safely.

I designed the application to meet all the final project requirements. It includes three main parts:

* **Basic Functions:** Adding, completing, and deleting tasks.

* **Authentication:** A secure user registration, login, and logout system.

* **File Processing:** Saving all application data to the local computer automatically.

2. Special Features

To make my project better than a standard list, I added these distinctive features:

* **Multi-User System:** Different people can use this application on the same computer. Everyone logs in with their own username. User A cannot see User B's tasks.

* **Simple Text File (.txt) Storage:** I did not use a complex database. I used standard `.txt` text files to save data. The system saves passwords in `users.txt` and creates a special file for each user's tasks (like `Ahmet_tasks.txt`). This makes the data very easy to read and check.

* **"Mark as Completed" Feature:** When users finish a task, they do not have to delete it. They can click the "Complete" button. The application puts a checkmark `[✓]` at the beginning of the task to show it is finished.

3. Design and Technical Details

I built the architecture of the application in three main parts:

A. User Interface (UI) Design

I used JavaFX to make a clean and modern screen. I wrote the UI dynamically with code using `VBox` and `HBox` layouts. The application switches between two screens (`Scene`):

1. **Login Screen:** Contains text fields for username and a hidden password field.

2. **Main Task Screen:** Contains the user's personal task list. I used an `ObservableList` and connected it to a `ListView`. Because of this, when I add or delete a task, the screen updates automatically.

B. Event-Driven Programming

The application waits for the user's actions. I used Java Lambda expressions (`e -> { ... }`) for button clicks.

* **Error Prevention:** If the user writes a wrong password or forgets to select a task before deleting, my `showAlert()` method opens a small warning window. The application never crashes.

* **Safe Close (Edge Case):** If the user clicks the red "X" button to close the app without logging out, my `window.setOnCloseRequest(...)` method catches it and saves the tasks automatically.

C. File Processing (File I/O)

I used the `java.io` library to read and write local files.

* **Fast Login:** The `loadUsers()` method reads the `users.txt` file line by line using `BufferedReader`. It puts the data into a `HashMap`. I used a `HashMap` because it can find a username and check the password instantly in O(1) time.

* **Data Saving:** The `saveTasks()` method uses `BufferedWriter`. It takes all tasks and writes them line by line (`newLine()`) back to the user's text file.

4. Conclusion

In conclusion, this project is a complete and working software. I successfully used Object-Oriented Programming (OOP) rules, Event-Driven UI design with JavaFX, and persistent File I/O handling. It is a secure, multi-user task management tool.